

RETAILHUB DATA PLATFORM

PYSPARK STREAMING & MEDALLION LAKEHOUSE

End-to-End Architectural Deep-Dive, Dry-Run Analysis & Code Procedures

ENGINEERING CORE SUMMARY

Medallion Architecture: Bronze (Raw Append) -> Silver (Cleaned, Lineage, Deduplicated) -> Gold (Kimball Star Schema).
Dual Engine Processing: Structured Streaming for Ingestion & Real-Time AI Defense; Batch PySpark for Silver & Gold.
Cloud Native: Zero OS filesystem dependencies on AWS S3 (S3A), fault-tolerant checkpoints, RDS MySQL JDBC.
Guarantees: Idempotent replays, multi-layer window deduplication, physical Kafka lineage tracking, 100% reconciliation.

PLATFORM SPECIFICATIONS & METRICS

Component	Technology	Configuration / Strategy	Data Volume / SLA
Ingestion Stream	Spark Streaming	Kafka Pattern retail_*, 5s	Real-time (<5s lag)
Fraud Defense	FraudGuard + Gemini	foreachBatch, 15 rules, Boun	Sub-second evaluation
Silver Medallion	PySpark Batch	12 domain folders + Quaranti	Hourly/Daily Scheduled
Gold Lakehouse	Kimball Star Schema	4 Dimensions + 5 Fact tables	Analytics / BI ready
Cloud Storage	AWS S3 / S3A	Snappy Parquet with atomic s	Petabyte-scale Lake
Relational Store	AWS RDS MySQL	PySpark JDBC partition-paral	Operational Master
Orchestration	Apache Airflow	DAG DAG-driven dependencies	Automated pipelines

Author: Data & Cloud Infrastructure Engineering Team

Runtime: Apache Spark 4.2.0 | Apache Kafka 3.8 (KRaft) | AWS Cloud | Python 3.12

Status: Verified Production-Ready | Fully Cloud-Hardened

1. Medallion Lakehouse Architecture & Principles

The RetailHub data platform implements a cloud-hardened Medallion Architecture (Bronze -> Silver -> Gold) designed for high-throughput e-commerce operations. It decouples high-velocity event ingestion from downstream dimensional modeling, ensuring data immutability, auditability, and resilient incremental processing.

Core Engineering Principles & Architectural Invariants

- ? Bronze preserves the exact payload received from Kafka as raw event_json. No schema mutations, no parsing, and no filtering are performed at the ingestion boundary. Raw Immutability (Bronze):
- ? Every record carries physical Kafka coordinates (topic, partition, offset). These coordinates serve as a physical primary key to drive idempotent anti-joins across pipeline runs. Physical Lineage Tracking:
- ? Deduplication occurs at two boundaries: current-batch Window functions (partitioned by event_id, ordered by offset desc) and cross-batch state anti-joins against existing Silver/Gold tables. Multi-Layer Deduplication:
- ? Dirty, malformed, or corrupt records are never dropped silently. They are tagged with specific validation error codes and routed to a dedicated Quarantine S3 prefix for debugging and replay. Strict Quarantine Isolation:
- ? The Gold layer structures data into conformant dimensions and business lifecycle fact tables, supporting fast OLAP analytics, Athena querying, and BI dashboards. Kimball Star Schema (Gold):

End-to-End Medallion Data Flow Blueprint

```
+-----+
| KAFKA KRaft (12 Topics) ---> Structured Streaming (5s trigger) ---> S3 BRONZE (Parquet) |
|      |                                                                    |
|      +--> FraudGuard Engine (foreachBatch + 15 Rules + Gemini AI) ---> Kafka Alerts |
+-----+
      |
      v
+-----+
| SPARK BATCH (bronze_to_silver.py): Lineage Left-Anti Join, Schema Parsing, 25+ DQ Rules |
|      |                                                                    |
|      v (Valid Events)                v (Malformed / Duplicate)            |
| S3 SILVER (12 Domain Folders)        S3 QUARANTINE (Quarantine Parquet) |
+-----+
      |
      v
+-----+
| SPARK BATCH (Gold Aggregations): Star Schema (4 Dimensions + 5 Fact Tables) |
| RDS MySQL JDBC (dim_product, dim_customer) + Silver Event Lifecycle Aggregations |
| S3 GOLD ---> AWS Glue Catalog ---> Amazon Athena ---> Executive BI & Analytics |
+-----+
```

WHY NOT WRITE DIRECTLY TO SILVER OR DELTA LAKE?

1. Decoupled Blast Radius: Streaming writing directly to Silver introduces tight coupling. If validation rules change, reprocessing requires re-consuming
2. Cost Optimization: Standard Parquet on S3 avoids expensive Delta/Iceberg transaction log coordination on basic EC2 setups while delivering identical
3. Immutability & Compliance: Bronze serves as an immutable write-once read-many audit trail of all website interactions.

2. Real-Time Streaming Ingestion (Kafka -> Bronze)

The file `src/streaming/kafka_to_bronze.py` establishes the first ingestion boundary. It uses Apache Spark Structured Streaming to read across all RetailHub Kafka topics simultaneously, attaching physical Kafka metadata and committing raw Parquet micro-batches to Amazon S3.

Key Methods & Configurations Used in `kafka_to_bronze.py`

- ? Initializes the Structured Streaming `DataFrameReader`. Subscribes using topic pattern `retail_*` which dynamically discovers all 12 operational topics without restart. `readStream.format('kafka')`:
- ? Ensures predictable consumer group offset initialization. In production, `failOnDataLoss` is set to `false` to tolerate Kafka topic retention truncation. `startingOffsets='latest' / 'earliest'`:
- ? Extracts physical coordinates: `col('topic')`, `col('partition')`, `col('offset')`, `col('timestamp').alias('kafka_timestamp')`, and key/value string casting. Kafka Metadata Projection:
- ? Appends `bronze_ingestion_time` via `current_timestamp()` to capture system landing latency independently from producer `event_time`. Ingestion Time Injection:
- ? Micro-batch trigger providing near-real-time S3 delivery without overwhelming S3 with excessive microscopic file objects. `Trigger(processingTime='5 seconds')`:
- ? Stores streaming state in S3 `BRONZE_CHECKPOINT_PATH`. Enables exactly-once fault recovery if container or EC2 restarts. Checkpoint Location:

Bronze Projection & Storage Structure

```
# Exact Projection executed by kafka_to_bronze.py:
kafka_events = raw_stream.select(
    col('topic'),
    col('partition'),
    col('offset'),
    col('timestamp').alias('kafka_timestamp'),
    col('key').cast('string').alias('kafka_key'),
    col('value').cast('string').alias('event_json')
).withColumn('bronze_ingestion_time', current_timestamp())

# FileStreamSink Commit to S3:
query = bronze_events.writeStream \
    .format('parquet') \
    .outputMode('append') \
    .option('path', BRONZE_PATH) \
    .option('checkpointLocation', BRONZE_CHECKPOINT_PATH) \
    .trigger(processingTime='5 seconds') \
    .start()
```

Bronze Schema Specification (Enforced at Silver Read)

Field Name	PySpark Data Type	Nullable	Description / Purpose
topic	StringType	True	Origin Kafka topic name (e.g. retail_order)
partition	IntegerType	True	Kafka partition number (0, 1, 2)
offset	LongType	True	Kafka 64-bit integer monotonically increases
kafka_timestamp	TimestampType	True	Broker record append timestamp
kafka_key	StringType	True	Partitioning key (e.g. customer_id or sess)
event_json	StringType	True	Original complete raw JSON payload (immutable)
bronze_ingestion_time	TimestampType	True	Spark cluster micro-batch processing times

3. Real-Time AI Fraud Defense (FraudGuard)

Operating concurrently with Bronze ingestion, `src/streaming/fraud_consumer.py` provides sub-second fraud and security analytics. It employs a high-throughput `foreachBatch` micro-batch pattern coupled with bounded in-memory state tracking to evaluate 15 deterministic fraud rules and trigger Gemini AI explanations.

Core Processing Architecture in `fraud_consumer.py`

- ? Consumes Kafka topics with a single `readStream` and a single `foreachBatch` action, preventing repeated Kafka consumer rebalances. Single Stream Pattern:
- ? Sets `maxOffsetsPerTrigger=5000` to cap batch size, preventing cluster out-of-memory errors during traffic spikes. Backpressure Management:
- ? Maintains thread-safe sliding state dicts bounded by Time-To-Live (TTL) cleanup windows (e.g. 5m, 15m, 1h), avoiding heavy Spark StateStore disk overhead. Bounded In-Memory State:
- ? Uses a single `toLocalIterator()` traversal over batch rows instead of `count() + collect()`, slashing driver CPU overhead. Zero Repetitive Actions:

The 15 Deterministic Fraud Detection Rules

Rule ID	Rule Name	Window / Threshold	Detection Logic & Mechanics
R01	CARD_TESTING	3 failed in 5 min	Rapid payment failure bursts indicative
R02	PAYMENT_VELOCITY	> 4 attempts in 10 mi	Abnormal transaction attempt frequency
R03	MULTI_ACCOUNT_IP	> 3 accounts per IP	Multiple disparate customer accounts tr
R04	ACCOUNT_TAKEOVER	Fail login + pass cha	Failed authentication sequence followed
R05	NEW_DEVICE_SENSITIVE	Unknown device + prof	Profile/address change on device never
R06	COUPON_ABUSE	> 5 uses across accou	Single discount voucher applied across
R07	HIGH_VALUE_BURST	Order > \$1,000 & 2x a	High-value order volume deviating drast
R08	IMPOSSIBLE_TRAVEL	Speed > 800 km/h	Physical geographic distance between co
R09	REFUND_ABUSE	> 3 returns in 7 days	High ratio of return requests to comple
R10	SESSION_HIJACK	IP change mid-session	Session token reused from different sub
R11	BOT_CART_HOARDING	> 20 items in 60 sec	Rapid cart reservations designed to exh
R12	ADDRESS_MISMATCH	Billing != Shipping s	Discrepancy between card issuer country
R13	OFF_HOURS_ADMIN	Admin login 00:00-05:	Administrative privilege escalation dur
R14	CHARGEBACK_CLUSTER	Device shared w/ bad	Device fingerprint linked to previously
R15	SYNTHETIC_IDENTITY	Disposable email + VO	Temporary email domain paired with virt

GEMINI AI FRAUD EXPLAINER INTEGRATION

When high-severity alerts trigger (e.g. Card Testing + Impossible Travel), FraudGuard formats a contextual prompt containing historical user metadata and latency vectors, calling Google Gemini 2.5 Flash asynchronously. Gemini generates a structured natural language risk synopsis, confidence score (0-100), and remediation recommendation which is emitted to the `retail_fraud_alerts` Kafka topic and persisted to disk.

4. Silver Processing (Cleansing, Lineage & Deduplication)

The core data engineering logic resides in `src/silver/bronze_to_silver.py`. This batch pipeline transforms raw, untyped Bronze Parquet files into clean, validated, normalized, and deduplicated Silver domain datasets while isolating corrupt events into Quarantine.

Step-by-Step Execution Mechanics in `bronze_to_silver.py`

- ? Reads `s3a://.../bronze` using explicit `BRONZE_SCHEMA`. If the directory is empty, `count()` returns 0 and the pipeline exits gracefully without throwing `UNABLE_TO_INFER_SCHEMA`. 1. Resilient Bronze Reading:
- ? Reads lineage (topic, partition, offset) from all 12 Silver domain folders plus Quarantine. Joins against Bronze via `left_anti` to extract ONLY new/unprocessed records. 2. Incremental Lineage Anti-Join:
- ? Executes `F.from_json(event_json, event_schema)`. Extracts nested context (device, IP, geo), entity (foreign keys), and metadata (amounts, prices, ratings). 3. Deep JSON Schema Parsing:
- ? Converts ISO strings via `F.try_to_timestamp()`. Computes `event_arrival_lag_seconds` and `producer_ingestion_lag_seconds` to detect clock skew and transmission delays. 4. Timestamp & Latency Calculation:
- ? Applies `Window.partitionBy('event_id').orderBy(kafka_timestamp.desc(), offset.desc())`. Flags records where `_row_number > 1` as `DUPLICATE_EVENT_ID`. Joins with existing Silver `event_ids` to stop cross-run duplicates. 5. Multi-Layer Deduplication Engine:
- ? Validates mandatory keys per `event_type`. Checks numeric boundaries (negative prices, invalid quantities, ratings not in 1..5). Flagged rows receive `quarantine_reason`. 6. 25+ Business DQ Rules:
- ? Splits dataset: rows with `quarantine_reason IS NULL` route to `valid_silver`; rows with `quarantine_reason IS NOT NULL` route to quarantine. 7. Dual-Path Partition Routing:
- ? Valid silver events are filtered and written in append mode to 12 respective S3 folders (`order_events`, `payment_events`, `delivery_events`, etc.). 8. Domain Partitioned S3 Writing:
- ? Asserts `valid_count + quarantine_count == new_bronze_count`. If reconciliation fails, raises a fatal exception to halt processing. 9. Reconciliation Audit Invariant:

Silver Lineage & Deduplication Window Logic

```
# Step 1: Incremental Lineage Anti-Join
bronze_new = bronze.alias('b').join(
    processed_keys.alias('p'),
    (F.col('b.topic') == F.col('p.topic')) &
    (F.col('b.partition') == F.col('p.partition')) &
    (F.col('b.offset') == F.col('p.offset')),
    'left_anti'
)

# Step 2: Multi-Layer Current-Batch Deduplication Window
window_spec = Window.partitionBy('event_id').orderBy(
    F.col('kafka_timestamp').desc_nulls_last(),
    F.col('partition').desc(),
    F.col('offset').desc()
)
with_duplicates = validated.withColumn('_row_number', F.row_number().over(window_spec)) \
    .withColumn('quarantine_reason', F.when(F.col('_row_number') > 1, 'DUPLICATE_EVENT_ID')
        .otherwise(F.col('quarantine_reason')))
```

Silver Routing Mapping: Kafka Topics to S3 Storage Folders

Kafka Source Topic	Silver Destination Folder	Primary Event Types Contained
retail_order_events	order_events	order_created, order_confirmed, order_c
retail_payment_events	payment_events	payment_initiated, payment_retry, payme
retail_fulfillment_events	fulfillment_events	inventory_reserved, inventory_released,

retail_delivery_events	delivery_events	in_transit, out_for_delivery, delivered
retail_cart_events	cart_events	cart_item_added, cart_item_removed, car
retail_discovery_events	discovery_events	product_view, product_impression, searc
retail_checkout_events	checkout_events	checkout_started, checkout_abandoned, a
retail_coupon_events	coupon_events	coupon_viewed, coupon_applied, coupon_r
retail_return_events	return_events	return_requested, return_approved, retu
retail_user_events	user_events	login, logout, session_started, session
retail_admin_events	admin_events	admin_login, admin_dashboard_viewed
retail_system_events	system_events	invoice_generated, notification_created

5. Gold Dimensional Modeling (Kimball Star Schema)

The Gold layer transforms event-level Silver datasets into a Kimball-style Star Schema optimized for business intelligence, reporting, and executive metrics. It comprises 4 Dimension tables and 5 Fact tables, combining MySQL operational relational master data with Silver event history.

The 4 Dimension Tables (src/gold/dim_*.py)

? Deterministically generates a continuous 10-year calendar range (2020-01-01 to 2030-12-31) in PySpark. Computes date_key (YYYYMMDD integer), year, quarter, month, day, day_name, day_of_week, is_weekend, and fiscal quarters. dim_date (dim_date.py):

? Reads products master table from AWS RDS MySQL via Spark JDBC. Enforces SCD Type 1 attributes: product_id, sku, product_name, category_id, brand, base_price, cost_price, is_active. Validates zero null primary keys. dim_product (dim_product.py):

? Reads customers table from AWS RDS MySQL via Spark JDBC. Transforms demographic records: customer_id, full_name, email_hash, city, state, country, registration_date, customer_segment. Sanitizes PII. dim_customer (dim_customer.py):

? Reads categories and subcategories from MySQL via JDBC. Joins category hierarchy: category_id, category_name, parent_category_id, subcategory_id, subcategory_name, commission_rate. dim_category (dim_category.py):

The 5 Fact Tables (src/gold/fact_*.py)

? Grain: One row per order. Multi-domain join across order_events, payment_events, fulfillment_events, delivery_events. Computes cycle latency (order_to_payment_seconds, order_to_ship_seconds, order_to_delivery_seconds) and financial balances. fact_orders (fact_orders.py):

? Grain: One row per order line item. Integrates cart, checkout, order, fulfillment, delivery, and return events. Computes line item prices, item discount allocations, tax portions, item fulfillment statuses, and return tracking. fact_order_items (fact_order_items.py):

? Grain: One row per payment transaction attempt. Reconstructs payment state machine: initiated -> retry -> success/failed. Validates payment chronology, payment gateways, card brands, currencies, and refund balances. fact_payments (fact_payments.py):

? Grain: One row per shipment / delivery package. Tracks fulfillment milestones: label_created, in_transit, out_for_delivery, delivered, delivery_failed. Computes carrier performance, transit hours, delivery attempt count, and SLA adherence. fact_delivery (fact_delivery.py):

? Grain: One row per user product interaction. Combines discovery_events (searches, impressions, views) and cart_events. Evaluates clickstream conversion funnels, search rank positions, CTR, and cart abandonment. fact_product_interactions (fact_product_interactions.py):

Gold Fact Table Schema Specifications

Fact Table	Primary Key	Foreign Keys (Dims)	Core Measures & Metrics
fact_orders	order_id	customer_id, date_key, add	total_amount, subtotal, tax,
fact_order_items	order_item_id	order_id, product_id, cust	quantity, unit_price, line_to
fact_payments	payment_id	order_id, customer_id, dat	amount, payment_latency_sec,
fact_delivery	shipment_id	order_id, carrier_id, date	transit_hours, delivery_attem
fact_product_interacti	event_id	product_id, customer_id, d	result_count, search_position

6. End-to-End Operational Dry-Run (Step-by-Step Simulation)

To understand how the code behaves during live execution, this section traces a single simulated e-commerce order (\$150 purchase of a Wireless Headset by customer CUST_8842) through every stage of the platform from Kafka generation to Gold Star Schema analytics.

Simulation Trace: The Lifecycle of Order ORD-99104

? Customer places an order on the website. Frontend publishes order_created event to retail_order_events with event_id=EVT_001, order_id=ORD-99104, total=150.00, customer_id=CUST_8842, partition=1, offset=4120. T+00s | Event Emission (Kafka):

? fraud_consumer.py receives EVT_001. Evaluates velocity and payment history in bounded memory. All rules pass. Emits zero fraud flags. Batch completes. T+02s | FraudGuard Stream Processing:

? kafka_to_bronze.py trigger fires. Extracts topic, partition=1, offset=4120, kafka_timestamp=2026-10-05 12:00:00, appends bronze_ingestion_time=2026-10-05 12:00:05. Writes Parquet micro-batch to S3 Bronze. T+05s | Structured Streaming Ingestion:

? bronze_to_silver.py runs via Airflow. Scans existing lineage; detects offset 4120 is missing from Silver. Anti-join retains EVT_001. Parses JSON schema. Calculates arrival lag = 5s. Validates order schema rules. Window deduplication assigns _row_number=1. Evaluates quarantine: quarantine_reason is NULL. Routes EVT_001 to S3 s3a://.../silver/order_events. Reconciliation check: 1 read = 1 valid + 0 quarantine. PASS. T+15m | Bronze to Silver Batch Execution:

? Subsequent events payment_success (offset 5102), order_shipped (offset 6011), and delivered (offset 7200) arrive in Kafka and are processed to Bronze and Silver in identical fashion. T+30m | Payment & Shipping Events Stream:

? fact_orders.py runs. Detects new lineage for ORD-99104 across order, payment, fulfillment, and delivery Silver domains. Flags ORD-99104 as affected. Pulls ALL historical Silver events for ORD-99104. Computes order_to_payment_seconds=34s, order_to_ship_seconds=3600s, order_to_delivery_seconds=86400s. Validates financial consistency: \$150.00 == \$135.00 + \$15.00 tax. Existing Gold is cached in memory. Writes rebuilt row to S3 Gold. Completes. T+01h | Gold fact_orders Rebuild:

Operational Dry-Run State Transition Matrix

Stage	Engine / Script	Input Dataset	Output Dataset / Artifact	Guarantees
Ingestion	kafka_to_bronze.py	Kafka retail_*	S3 Bronze Parquet	Append-onl
AI Defense	fraud_consumer.py	Kafka Stream	Kafka Alerts & Incident	Bounded TT
Cleansing	bronze_to_silver.py	S3 Bronze Parquet	S3 Silver (12 Folders)	Offset Ant
Dimensions	dim_*.py	RDS MySQL JDBC	S3 Gold dim_* Tables	SCD Type 1
Fact Engine	fact_orders.py	4 Silver Domains	S3 Gold fact_orders Tab	Rebuild Af
Line Item	fact_order_items.py	6 Silver Domains	S3 Gold fact_order_item	Grain: 1 R
Logistics	fact_delivery.py	2 Silver Domains	S3 Gold fact_delivery T	Carrier SL

7. Technical Inventory & Metrics ('How and How Many')

A rigorous quantification of the RetailHub data platform architecture, detailing exact counts of Kafka topics, event types, PySpark transformations, window operations, and S3 data lake storage paths.

Exact Numerical Inventory

? retail_admin_events, retail_cart_events, retail_checkout_events, retail_coupon_events, retail_delivery_events, retail_discovery_events, retail_fulfillment_events, retail_order_events, retail_payment_events, retail_return_events, retail_system_events, retail_user_events. Kafka Topics (12 Primary):

? Spans 12 domains: 4 Order lifecycle, 7 Payment/Refund, 5 Fulfillment, 4 Delivery, 3 Cart, 12 Discovery/Search, 8 Checkout/Address, 4 Coupon, 5 Return, 5 User/Session, 2 Admin, 4 System/Notification. E-Commerce Event Types (60+):

? F.col, F.lit, F.when, F.current_timestamp, F.from_json, F.try_to_timestamp, F.unix_timestamp, F.row_number, F.sum, F.count, F.avg, F.min, F.max, F.round, F.coalesce, F.trim, F.upper, F.lower, F.isnan, F.isnull, F.desc, F.asc, F.distinct, F.dropDuplicates, F.unionByName. PySpark Built-in Functions Used (25+):

? 1. Current-batch Event ID Deduplication; 2. Order Created Authoritative Attribute Selection; 3. Order Status Chronological Sequencing; 4. Payment Attempt Monotonic Numbering; 5. Delivery Transit Interval Calculation. Window Specifications (5 Advanced Specifications):

? 4 Dimension Tables (dim_date, dim_product, dim_customer, dim_category) + 5 Fact Tables (fact_orders, fact_order_items, fact_payments, fact_delivery, fact_product_interactions). Gold Star Schema Tables (9 Tables):

? MISSING_EVENT_ID, CORRUPT_JSON, UNPARSEABLE_EVENT_TIME, DUPLICATE_EVENT_ID, CROSS_RUN_DUPLICATE, MISSING_ORDER_ID, MISSING_CUSTOMER_ID, MISSING_PRODUCT_ID, MISSING_SESSION_ID, NEGATIVE_PRICE, NEGATIVE_QUANTITY, INVALID_RATING, INVALID_TAX_AMOUNT, INVALID_SHIPPING_FEE, INVALID_COUPON_DISCOUNT, etc. Quarantine Reason Codes (25+ Codes):

PySpark Methods & Architectural Role Mapping

PySpark Method / API	Pipeline Layer	Architectural Role & Significance
readStream.format('kafka')	Streaming	Subscribes to distributed Kafka topic stream wit
writeStream.outputMode('a	Streaming	Appends immutable raw micro-batches to S3 Bronze
foreachBatch(process_batc	Streaming (Fraud)	Applies micro-batch bounded memory state engine
spark.read.schema(SCHEMA)	Silver / Gold	Explicit schema reading eliminates expensive foo
join(..., how='left_anti'	Silver & Facts	Filters previously processed physical Kafka offs
from_json(col, schema)	Silver	Parses deep nested event JSON structures into ty
Window.partitionBy().orde	Silver & Facts	Computes analytical row numbers, lifecycle seque
df.cache() / df.count()	Gold Facts	Materializes DataFrames in executor memory to al
df.write.mode('overwrite'	Gold Facts	Natively cleans and writes S3 Parquet partitions
read.format('jdbc')	Gold Dimensions	Parallel partition read from AWS RDS MySQL maste

8. Cloud Operations, Airflow Orchestration & Tuning

Deploying PySpark Structured Streaming and Batch workloads on AWS requires specific tuning around cloud storage connectors, memory allocation, and workflow orchestration via Apache Airflow.

Apache Airflow DAG Scheduling Architecture

Batch transformations are scheduled as dependency DAGs in Airflow running inside dockerized workers on EC2. The gold_dag.py coordinates dimensional dependencies before triggering Fact lifecycle recalculations:

```
# Airflow Execution Graph:
silver_batch_task >> [dim_date_task, dim_product_task, dim_customer_task, dim_category_task]
[dim_product_task, dim_customer_task] >> fact_orders_task
fact_orders_task >> [fact_order_items_task, fact_payments_task, fact_delivery_task]
silver_batch_task >> fact_product_interactions_task
```

Key Performance & Reliability Optimizations Implemented

- ? Resolved SparkException: s3a://bucket is not absolute path by appending subpaths (s3a://bucket/events) and normalizing all paths with .rstrip('/') to avoid double slashes (s3a://bucket//folder). 1. S3A Absolute Path Normalization:
- ? Replaced glob.glob(), os.path.exists(), pathlib.Path.exists(), and shutil.rmtree() with native Spark S3A operations. Python OS calls only inspect local disk and fail silently on S3. 2. Eliminated Python OS Calls on S3:
- ? Replaced schema inference with explicit StructType schemas. When directories contain 0 records or empty Parquet footers, Spark counts 0 rows and exits cleanly instead of throwing UNABLE_TO_INFER_SCHEMA. 3. Zero-Record Schema Resilience:
- ? In Gold fact tables that read and overwrite the same S3 directory (e.g. fact_orders), df.cache() + df.count() materializes existing data in memory before writing, eliminating FileNotFoundError stage failures. 4. Lazy Evaluation Overwrite Deadlock Prevention:
- ? Set spark.sql.shuffle.partitions=8 across all jobs (default is 200). Eliminates severe shuffle partition fragmentation and CPU task overhead on medium-sized EC2 instances. 5. Spark Shuffle Partition Tuning:
- ? Stripped 450MB pip-reinstalled PySpark binaries from the base apache/spark:4.2.0-python3 image, reducing build time and container startup latency. 6. Lean Container Docker Images:

PRODUCTION DEPLOYMENT RUNBOOK (CLI COMMANDS)

1. Start Kafka & Streaming Ingestion on EC2:


```
docker compose -f src/streaming/docker-compose.yml up -d
```
2. Trigger Silver Cleansing Batch:


```
docker run --rm --env-file /opt/retailhub-batch/.env naveen9200/retailhub-spark-batch:4.2.0 \
  spark-submit /opt/retailhub/src/silver/bronze_to_silver.py
```
3. Trigger Gold Fact Orders Batch:


```
docker run --rm --env-file /opt/retailhub-batch/.env naveen9200/retailhub-spark-batch:4.2.0 \
  spark-submit /opt/retailhub/src/gold/fact_orders.py
```